GarantiFi — Technical Build Brief

For: Edson (Lead Developer) | Deadline: May 8, 2026 | Hackathon Submission: May 11, 2026

Context

We are submitting to the Colosseum Frontier Hackathon on Solana. Submission deadline is May 11, 2026 at 11:59pm PT. The submission requires three things from the technical side: (1) a working GitHub repository with code committed after April 6, (2) a recorded demo video showing the product working on Devnet, and (3) a live project link (dApp or landing page). Judges evaluate: functionality of the code, real-world impact, and novelty. Slides without working code will not be competitive.

The protocol is a collateralized lending system on Solana. Borrowers deposit physical luxury assets (watches, jewelry) with a custodian. Those assets are represented on-chain as compressed NFTs (TRDC tokens). Against those tokens, the protocol disburses stablecoin loans. The full technical specification is in garantifi-spec-v31-en-final.docx. This document tells you what to build, in what order, and what to skip before May 11.

Priority 1 — The Minimum Viable Demo (Must Ship)

The demo video must show one complete loan lifecycle on Devnet. Five instructions must work end-to-end, in this sequence:

1. initialize_vault

Deploys the VaultState account. This is the protocol's liquidity pool. It holds the stablecoin capital that gets disbursed to borrowers.

Required parameters:

- token_mint: use the standard Devnet USDC mint (Gh9ZwEmdLJ8DscKNTkTqPbNwLNNBjuSzaG9Vp2KGtKJr)

- min_ccb_amount: minimum loan amount (set to 1,000,000 for demo = 1 USDC with 6 decimals)

- scd_authority: a test keypair you control, acting as the licensed lending partner

> *Why it matters: the vault only accepts capital from the scd_authority. This is the legal separation — the protocol does not hold public deposits, the SCD (licensed lender) does. Judges with DeFi background will check this.*

2. deposit_capital

Called by the scd_authority keypair. Transfers test USDC from the SCD wallet into the vault's Associated Token Account (ATA).

IMPORTANT: track available liquidity using the live ATA balance (vault_ata.amount), NOT a cumulative formula like total_disbursed minus total_repaid. The spec is explicit on this — use the real balance.

> *Why it matters: demonstrates the SCD-gated capital model. Only the licensed partner puts money in.*

3. create_ccb_trdc

This is the core instruction. It mints a TRDC (compressed NFT via Metaplex Bubblegum) that represents an on-chain credit instrument linked to an off-chain loan agreement (CCB).

Required inputs:

- loan_amount: the amount to be disbursed

- ccb_hash: SHA-256 of a dummy PDF (you can generate one with any PDF and hash it)

- ccb_external_id: a dummy ID string representing the CCB number in the lender's system

- collateral_value_usd: appraised value of the physical asset

- due_ts: Unix timestamp for loan maturity

Validation logic to enforce:

- loan_amount >= vault.min_ccb_amount

- collateral_value * ltv_bps / 10000 >= loan_amount (LTV check, ltv_bps = 5000 for 50%)

- vault is not paused

- scd_pubkey in the call matches vault.scd_authority

Initial TRDC status: PENDING_KYC

> *For the demo: hardcode KYC as approved and immediately advance to PENDING_CUSTODY after creation. No real KYC bureau integration is needed — mock it with a status update.*
>
> *The cNFT should be visible in the borrower's Devnet wallet. Show this in the demo video — it is the visual proof that the physical asset is now represented on-chain.*

4. confirm_custody

Admin instruction. Triggered after the custodian confirms physical receipt of the asset. This is the instruction that transitions the TRDC from PENDING_CUSTODY to ACTIVE — and also fires the disbursement.

This instruction calls disburse_ccb on the Vault Program via CPI (Cross-Program Invocation). The key rule: money only leaves the vault AFTER this call. Not at createCCB. This separation is legally important and must be demonstrated clearly in the demo.

> *Why it matters: shows judges that the protocol cannot disburse funds until physical custody is confirmed. This is the bridge between the real world and the blockchain.*

5. repay_ccb

The borrower repays principal plus accrued interest. The TRDC status updates to REPAID. The cNFT can be burned or marked as closed. The custodian is notified off-chain to release the physical asset.

For the demo: calculate a simple interest amount inline. The full accrual formula is in the spec (Section 5.2) — use it so the math is auditable. Return the capital to the vault ATA.

> *DEMO SCRIPT: Two screens. Left: browser showing Devnet Explorer with each transaction hash. Right: dApp UI showing the loan status changing from PENDING to ACTIVE to REPAID. Total video: 3-4 minutes with narration.*

Priority 2 — Default and Renewal (Add If Time Allows)

If the Priority 1 demo is solid and there are days to spare before May 8, add these two instructions. They are differentiators — no other physical-collateral DeFi protocol has them working on-chain.

execute_af_default

Marks the TRDC as DEFAULTED. Does three things atomically: decrements vault.total_disbursed by the loan amount, increments vault.total_defaulted, and transfers the TRDC to a liquidation wallet. The actual physical asset seizure happens off-chain (Brazilian law DL 91.169 / Lei 14.711/2023). The on-chain part just locks the accounting and moves the token.

renew_ccb

The borrower pays only accrued interest, not the principal. The loan term extends. The TRDC status becomes RENEWED. The ccb_hash updates to the hash of the new amendment document. No new appraisal, no new logistics. This is the highest-margin operation in the model — the renewal cycle eliminates the ~665 BRL appraisal cost and raises gross margin from ~17% to ~26%. Show this working and explain it in the demo video.

Priority 3 — Security (Non-Negotiable Even for Hackathon)

These are not optional extras. Any DeFi judge will check for them. Without the CPI validation, any external program can drain 100% of the vault.

CPI Caller Validation

In both disburse_ccb and receive_repayment, validate that the caller is the Loan Program and nothing else. Pattern:

```
let expected = Pubkey::create_program_address( &[b"loan_program_authority",
&[bump]], &loan;_program_id, )?; require!(
ctx.accounts.caller_authority.key() == expected,
GarantiFiError::UnauthorizedCaller );
```

Integer Overflow Guards

Every arithmetic operation on u64 values uses checked_add / checked_sub / checked_mul. No .unwrap() on math operations — propagate errors with ?

pause_vault

Simple boolean flag (is_paused) in VaultState. Admin can set it. Every instruction that moves money checks this flag first and returns an error if paused. Needed for emergency stops and depeg events.

What NOT to Build Before May 11

These are in the spec but will not be finished in time and are not needed for the hackathon demo. Do not start them. Save time for a clean demo of Priority 1.

- Real KYC bureau integration (mock with a hardcoded APPROVED state)

- BRL stablecoin vault (USDC-only for the demo; the code supports any 6-decimal mint without changes)

- Depeg monitor (mock it as a config parameter)

- External SCD API (the SCD partner doesn't exist yet — mock the interface)

- Armored logistics integration (in-person custody model for MVP)

- Smart contract external audit (post-hackathon, budget ~R$80-150K)

- Full overdue cron (a manual markOverdue call in the demo is enough)

Tech Stack Summary

Smart contracts: Anchor / Rust on Solana

Two programs: Vault Program (VaultState) + Loan Program (TRDCAccount)

cNFT minting: Metaplex Bubblegum

Compressed NFT, under $0.001 per mint on Solana

Frontend: React + @solana/wallet-adapter

Basic UI is enough — focus on showing loan status changes

Metadata storage: IPFS or Arweave

Point to a dummy appraisal PDF for the demo — the URI goes in the cNFT metadata

Backend (mock): Node.js or FastAPI

Mock SCD API responses + custody webhook + simple overdue cron

Multisig: Squads Protocol

2/3 for admin operations (confirmCustody, pauseVault)

Devnet USDC: Gh9ZwEmdLJ8DscKNTkTqPbNwLNNBjuSzaG9Vp2KGtKJr

Standard Devnet USDC mint

GitHub Repository Requirements

Colosseum judges verify that code was created during the hackathon window (after April 6, 2026). Commit history must show substantive logic committed after April 6. Boilerplate scaffolding before that date is fine.

Required folder structure:

```
programs/ vault/ <- Anchor Vault Program (VaultState, deposit, disburse, pause) loan/ <- Anchor Loan Program (TRDCAccount, createCCB, confirmCustody, repay, default, renew) app/ <- React frontend backend/ <- Mock SCD service + overdue cron tests/ <- Full lifecycle test: init -> deposit -> createCCB -> confirmCustody -> disburse -> repay README.md <- Setup instructions + architecture diagram + demo video link
```

The test file is important. At least one end-to-end test that runs the full loan cycle. Judges look at the test suite to understand the protocol logic quickly.

Deadlines

April 21 — Confirm tech stack, start Vault Program skeleton

April 25 — initialize_vault + deposit_capital working on Devnet

April 30 — create_ccb_trdc + cNFT minting working on Devnet

May 4 — confirm_custody + disburse_ccb working on Devnet (full loan cycle live)

May 6 — repay_ccb working. Full cycle demo-able end-to-end

May 7 — renew_ccb + execute_af_default (if time allows). Record demo video

May 8 — Demo video delivered to George. GitHub repo clean and public

May 11 — Colosseum submission deadline — 11:59pm PT

Questions on the spec go to George. The full technical specification (garantifi-spec-v31-en-final.docx) has the complete data structures, all 17 TRDCAccount fields, the interest accrual formula, the 7-state TRDC machine, and the multi-token architecture. This document tells you what to build. The spec tells you exactly how.